Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

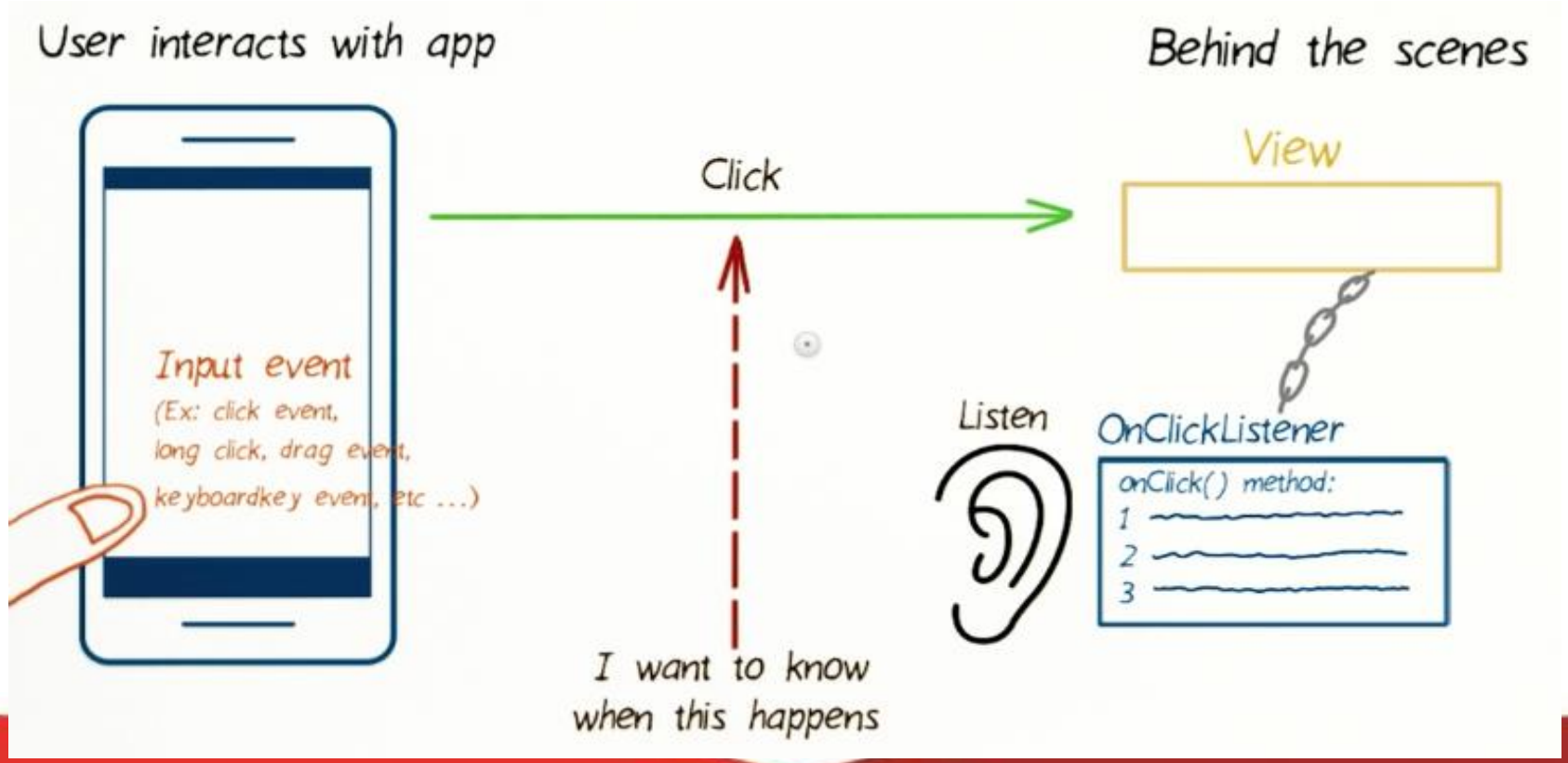
User Interaction

User Interaction

- Stateful & Stateless Widget
- Button
- SnackBar
- Dialog
- Menu
- Notification

Event Listener

- Like Swing, you can set the listener to the Widget



Why Use Event Listener

- To change the behavior of your button while your app is running. You can point your button at another method entirely, or just disable the button by setting it into doesn't do anything.

- To allows you to control a button's behavior from somewhere other than its dart page

Stateful & Stateless Widget

- Stateless Widget: or static **widget**, means the state of widget (layout and content) can't change and can't be changed

  Example: Icon, Text

- Stateful Widget: or dynamic **widget**, means the state of widget can change and can be changed

  Example: Checkbox, Radio, Slider, InkWell, Form, and TextField

Stateless Widget

Text with

a number 10

```
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('ExampleApp'),
      ),
      body: Center(
        child: Text(
          '10',
          style: TextStyle(
            fontSize: 30
          ),
        ),
      )
    );
  }
}
```

Stateful Widget

See the code when you create new project in Flutter. When we click the "+1" button, the number in the center will increment

```
class MyApp extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(
                title: Text('ExampleApp'),
            ),
            body: Container(
                child: NumberScreen(),
            )
        );
    }
}
```

Stateful Widget

```
class NumberScreen extends StatefulWidget {
  @override
  _NumberScreenState createState() =>
          _NumberScreenState();
}

class _NumberScreenState extends
                              State<NumberScreen> {
    int number = 10;
    @override
    Widget build(BuildContext context) {
        return Stack(
            children: <Widget>[
                Center(
                    child: Text(
                        this.number.toString(),
                        style: TextStyle(
                            fontSize: 30
```

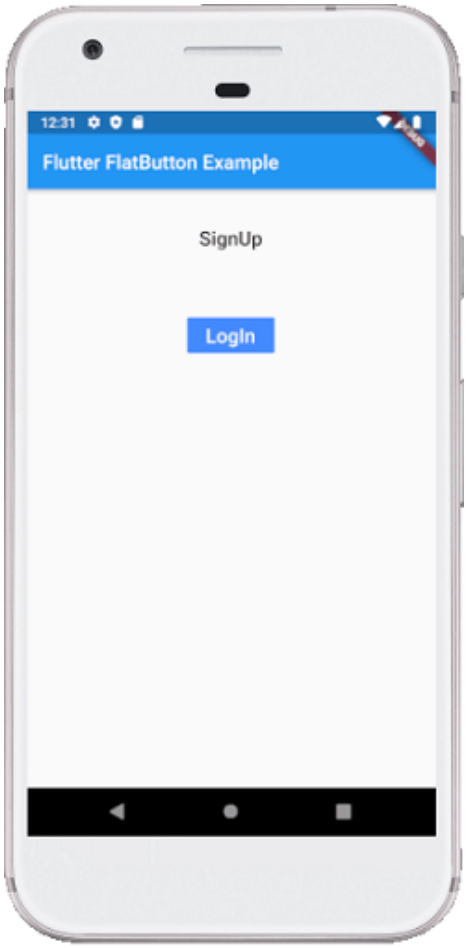
```
                        ),
                    ),
                ),
                Positioned(
                    bottom: 50,
                    right: 50,
                    child: FloatingActionButton(
                        child: Icon(Icons.plus_one),
                        onPressed: () {
                            setState(() {
                                this.number += 1;
                            });
                        },
                    ),
                )
            ],
        );
    }
}
```

Button

- Button is a widget that provides user to trigger an event
- There are many types of Button:
  - FlatButton
  - RaisedButton or ElevatedButton
  - OutlineButton
  - FloatingActionButton
  - DropdownButton
  - IconButton
  - PopupMenuButton

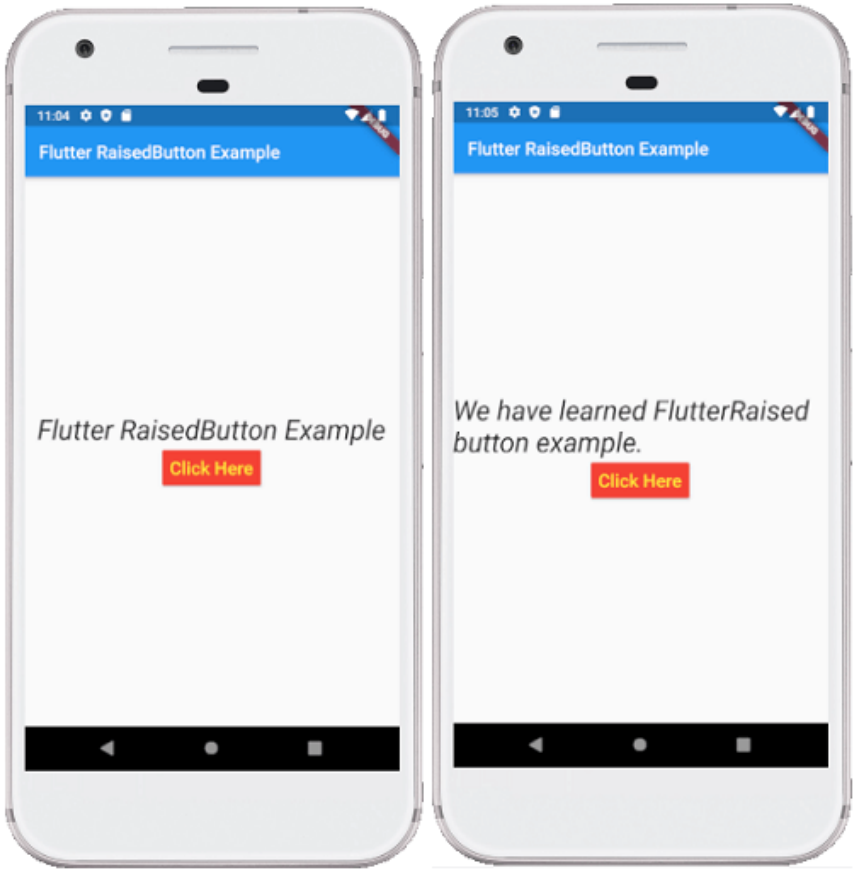
FlatButton

- Standard rectangle button with text. Can use color



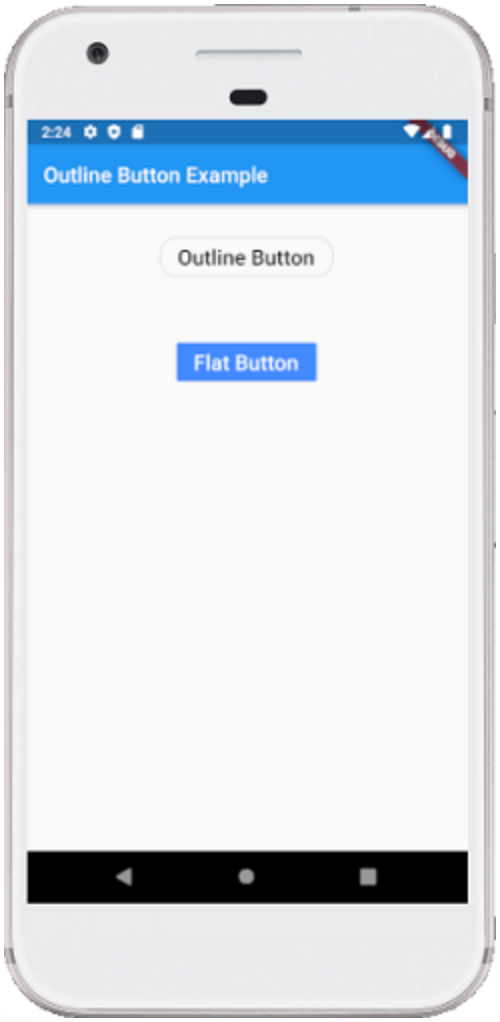
RaisedButton / ElevatedButton

- Like FlatButton, but has elevation (Z-axis change) when pressed. It has additional properties like shape, animation time, elevation



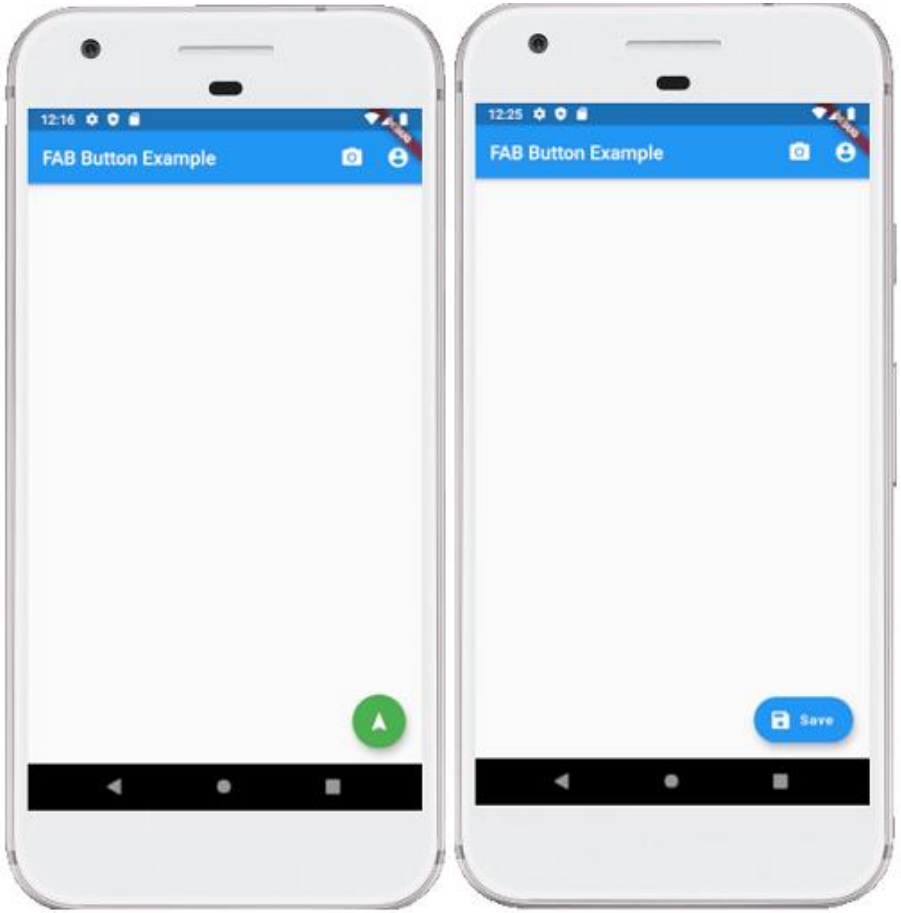
OutlineButton

- Like FlatButton, but has a thin grey rounded rectangle border



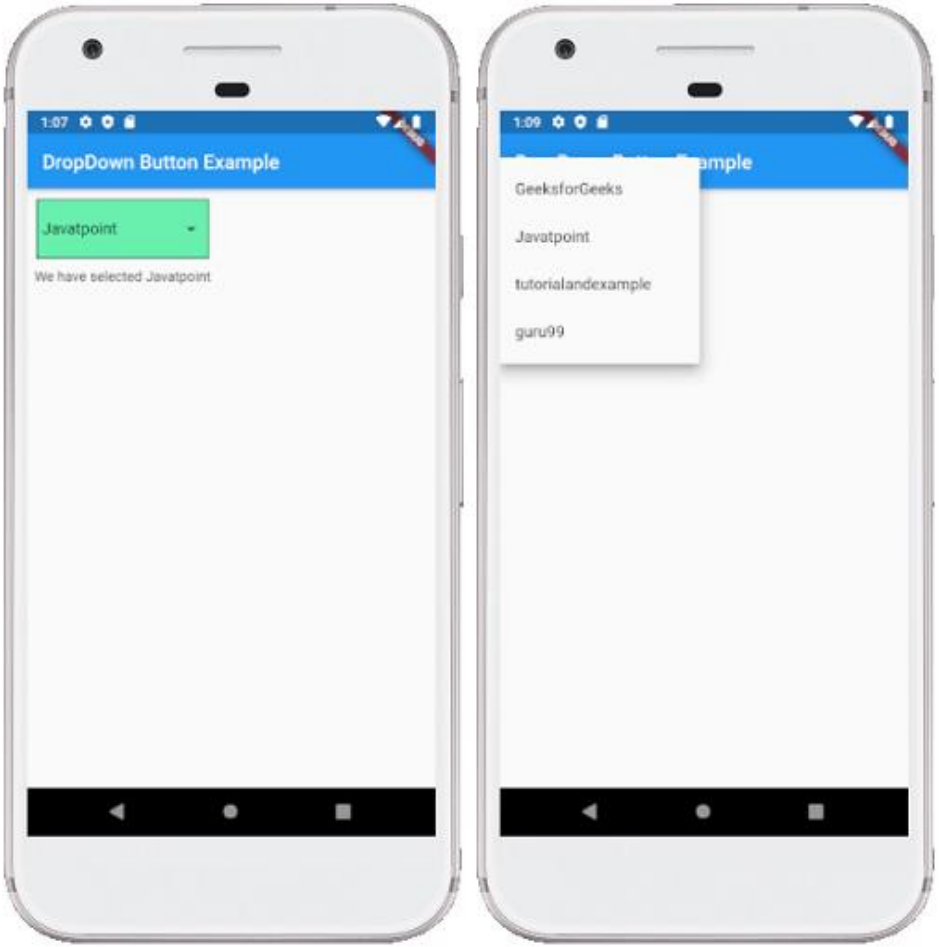
FloatingActionButton

- Circular icon button that floated and stacked on top of layout, usually positioned at bottom right corner. Can use image or icon



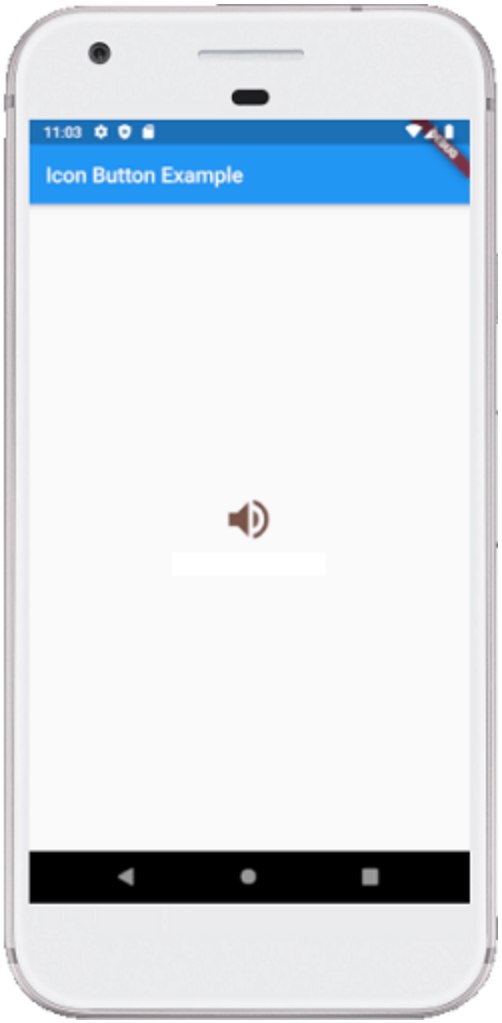
DropDownButton

- Button that contains overlay options



IconButton

- Button from Icon or Image



PopupMenuButton

- Button that displays menu when pressed. Usually used in AppBar

SnackBar

SnackBar is a floating rectangle widget that contains message and clickable text to do some action:

- Scaffold

```
return MaterialApp(
  title: 'SnackBar Demo',
  home: Scaffold(
    appBar: AppBar(
      title: const Text('SnackBar Demo'),
    ),
    body: const SnackBarPage(),
  ),
);
```

SnackBar

- Show the widget and call the action

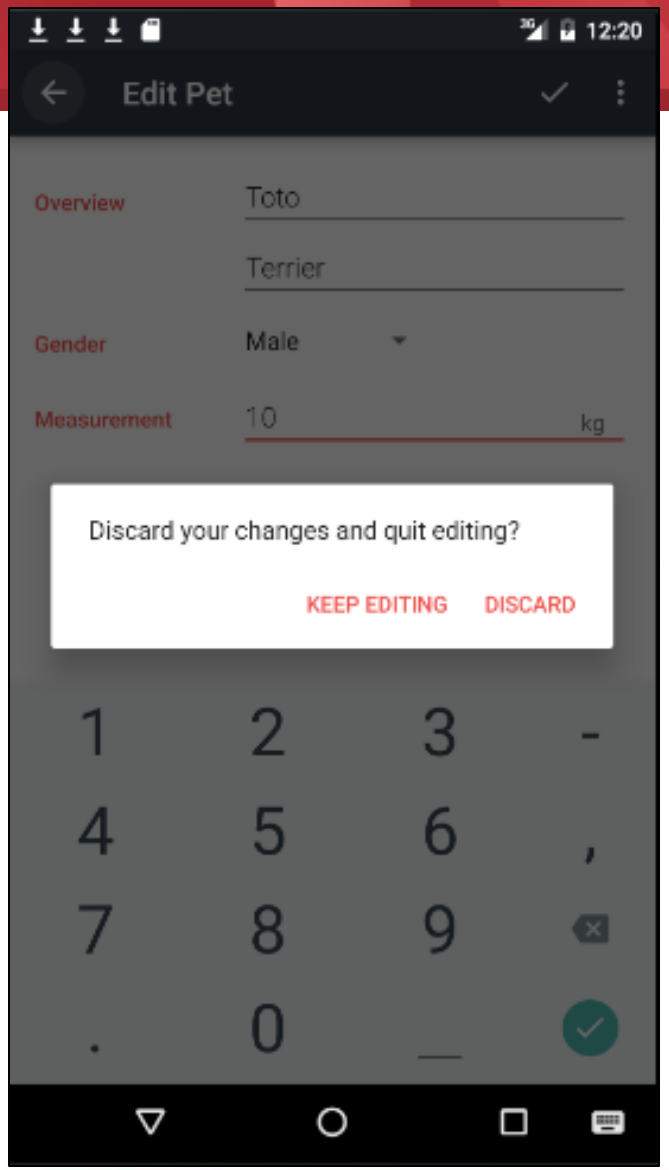
```
const snackBar = SnackBar(
  content: Text('Yay! A SnackBar!'),
);

// Find the ScaffoldMessenger in the widget tree
// and use it to show a SnackBar.
ScaffoldMessenger.of(context).showSnackBar(snackBar);
```

```
final snackBar = SnackBar(
  content: const Text('Yay! A SnackBar!'),
  action: SnackBarAction(
    label: 'Undo',
    onPressed: () {
      // Some code to undo the change.
    },
  ),
);
```

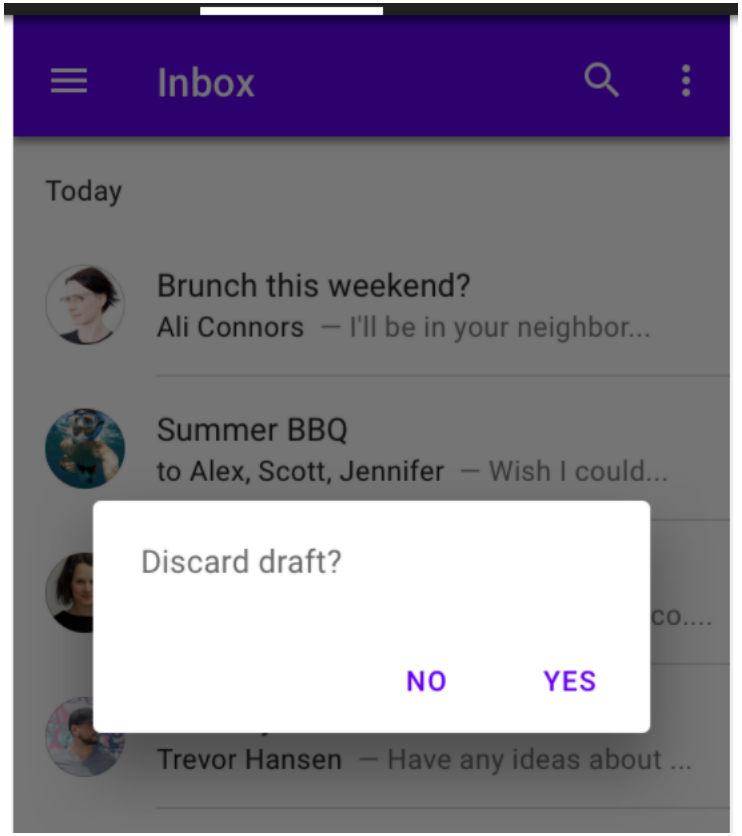
Dialog

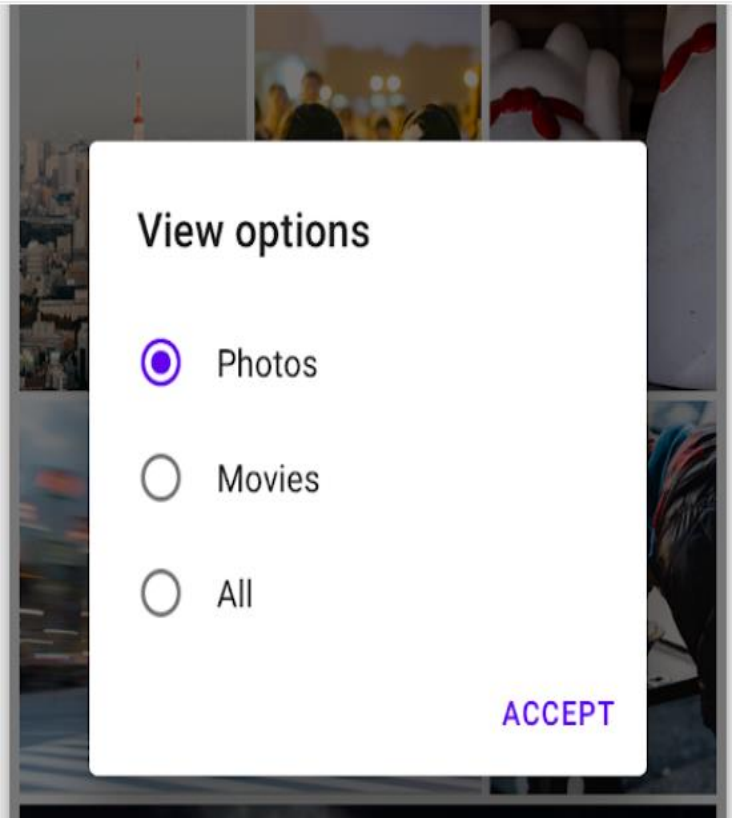
- A dialog is a small window that prompts the user to make a decision or enter additional information

- A dialog does not fill the screen and is normally used for modal events that require users to take an action before they can proceed.



Example: Alert Dialog & Simple Dialog





Alert Dialog

```
void tampilkanalert() {
  AlertDialog alertDialog = new AlertDialog(
    content: new Container(
      height: 200.0,
      child: new Center(
        child: new Text("Hello Ini Alert"),
      ),
    ),
    actions: [
      FlatButton(
        child: Text('Tutup'),
        onPressed: () {
          Navigator.of(context).pop();
        },
      ),
    ],
  );
  showDialog(context: context, child: alertDialog);
}
```

Simple Dialog

```
SimpleDialog simple = SimpleDialog(
  title: const Text('Select assignment'),
  children: <Widget>[
    SimpleDialogOption(
      onPressed: () { Navigator.pop(context, Department.treasury); },
      child: const Text('Treasury department'),
    ),
    SimpleDialogOption(
      onPressed: () { Navigator.pop(context, Department.state); },
      child: const Text('State department'),
    ),
  ],
);
```

Menu

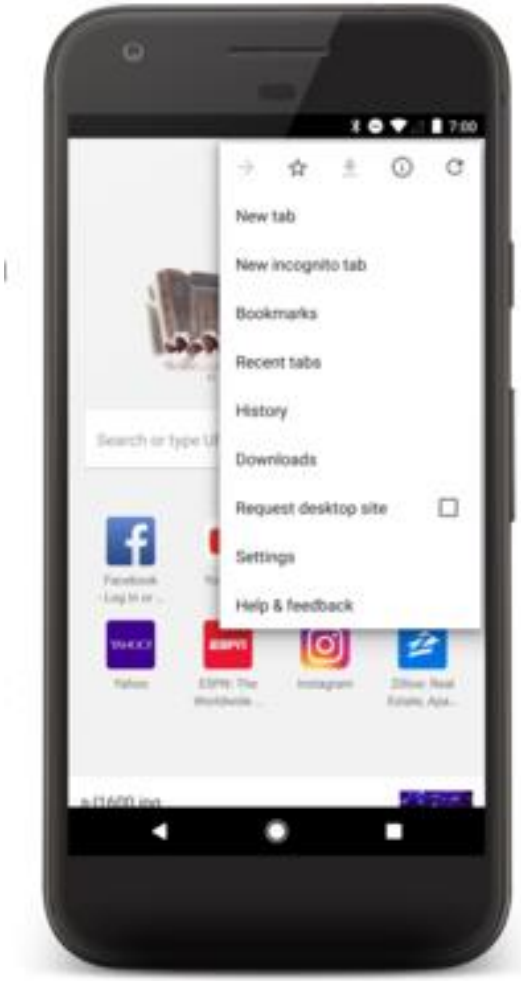
- TabBar (see previous chapter)
- BottomNavigationBar
  - Usually used for PageView
- AppBar

BottomNavigationBar

```
bottomNavigationBar: BottomNavigationBar(
  items: const <BottomNavigationBarItem>[
    BottomNavigationBarItem(
      icon: Icon(Icons.home),
      title: Text('Beranda'),
    ),
    ...
  ],
  currentIndex: 0,
  selectedItemColor: Colors.green,
  unselectedItemColor: Colors.grey,
  showUnselectedLabels: true,
),
```

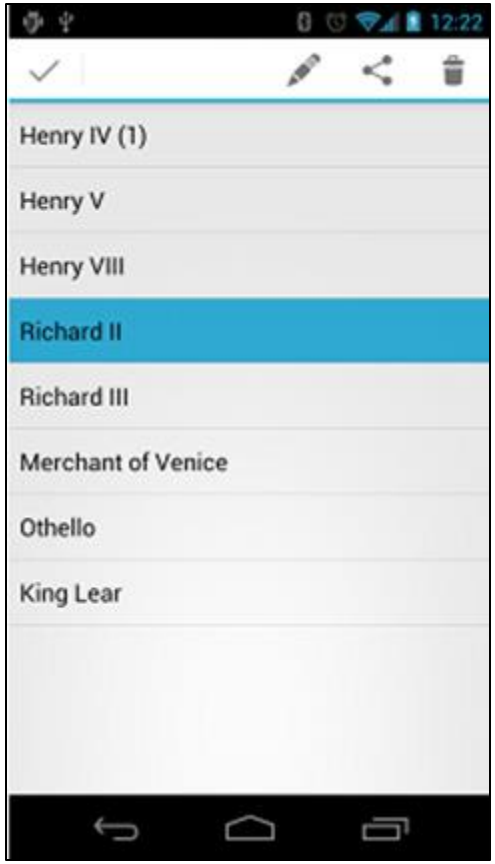
AppBar

4:20
Sheets


New tab
New incognito tab
Bookmarks
Recent tabs
History
Downloads
Request desktop site
Settings
Help & feedback

Contextual Menu: Action Mode

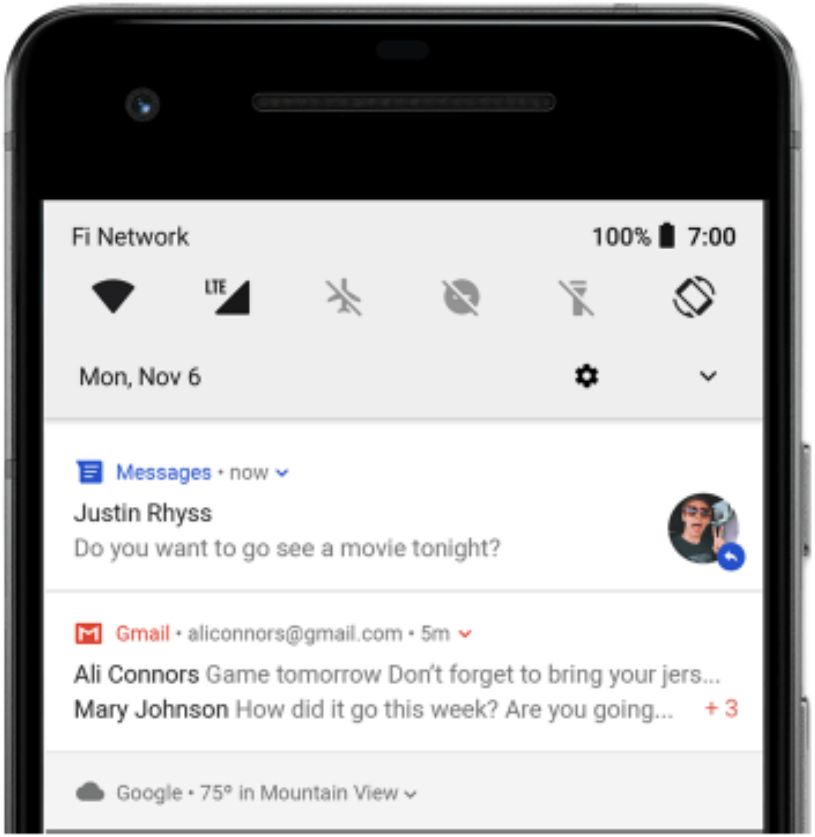
Displays action items that affect the selected content in a bar at the top of the screen and allows the user to select multiple items.



Invoke the contextual action mode upon one of two events (or both):

- User performs a long-click on the view.
- User selects a checkbox or similar UI component within the view.

Notification



Notification

- Notification is a useful and good feature for mobile app
- Notification uses a package

Local Notification VS Push Notification

- The major difference between local and push notifications is that local notifications are scheduled by an app locally and are delivered by the same device, whereas push notifications are sent from a remote server.

How To

- See:
  - https://pub.dev/packages/awesome_notifications
  - https://pub.dev/packages/flutter_local_notifications

Any question?

References

- https://docs.flutter.dev/development/ui/interactive
- https://docs.flutter.dev/development/ui/widgets/material
- https://www.javatpoint.com/flutter-buttons

Fakultas Informatika
School of Computing
Telkom University

THANK YOU